

Parallel and Distributed Computing

MPI Report

Juan Camilo Rodriguez Viteri Pedro Afonso Saraiva Gomes João Paulo Ferreira Russo
115766 103468 116543

Group 3

1 Introduction

This report is the follow-up to the previous OpenMP report. Its main objective is to parallelize the cabinet assignment using MPI. We showcase this through two different implemented strategies: Row-wise and Checkerboard decomposition. The performance of both strategies is evaluated through a series of experiments that analyze execution time, speedup, and scalability across different numbers of processes and cores.

2 Prior Version Implementation Changes

Prior to going to the main purpose of the report, we want to disclose some improvements we did to the Serial and OpenMP solutions.

2.1 Serial Implementation

An improvement that has been implemented has been to change how we were recomputing scores whenever documents are reassigned.

- **Version 1:** Stored document indexes per cabinet and recomputed cabinet scores from scratch after each reassignment.
- **Version 2:** Stored document counts and subject score sums per cabinet and recomputed cabinet scores using these stored values.
- **Version 3:** Updated cabinet scores incrementally by adding and subtracting document contributions during reassignment.

The latest and current has shown to be cheaper, Table 1 highlights performance over the different iterations.

Table 1: Serial Implementation Performance (Deucalion - large-arm partition)

Implementation	ex1000 (s)	ex10 (s)	ex200 (s)
Version 1	1059.4	1470.3	1386.4
Version 2	1059.2	1325.2	1423.5
Version 3	1058.9	1302.1	1307.2

2.2 OpenMP

The most contrasting change was the removal of paddings for all auxiliary arrays and the use of OMP's array reductions. The padded arrays were previously introduced to ensure no false-sharing happens when threads are updating values at the boundaries between per-thread arrays, since OMP does not give use that guarantee. We also now use *parallel* on each *for* instead, allowing us to remove the *single* and its implicit barrier.

Table 1: Run times and speedups for each input configuration (lab6p6).

Test	T	Old Time	Time	Speedup
ex1000	Serial	203.6	204.0	—
	1	204.1	203.3	1.00×
	2	103.8	103.4	1.97×
	4	52.5	52.3	3.90×
	6	35.6	35.4	5.76×
ex10	Serial	235.6	249.5	—
	1	264.2	235.7	1.06×
	2	135.4	119.8	2.08×
	4	70.3	60.5	4.12×
	6	52.0	41.2	6.06×
ex200	Serial	246.1	252.5	—
	1	251.1	246.5	1.02×
	2	131.3	125.3	2.01×
	4	66.5	63.2	4.00×
	6	46.2	43.0s	5.87×

With our new OpenMP implementation, we achieved better times for all problem instances, regardless of thread count. However, we noticed that even though our serial implementation was the best one we have achieved so far, that it shows abnormal times when compared to the OpenMP solution running on one single thread.

3 Parallelization Approach

3.1 Algorithm Characterization

The algorithm used to solve this problem is iterative, and with each iteration there are a number of calculations that need to be performed. The most computationally costly operation is to calculate the distance of each document with each cabinet. Overall, per iteration the computational cost is of:

$$T_{\text{iter}} = O(D \times C \times S) \quad (1)$$

where:

- D is the number of documents,
- C is the number of cabinets,
- S is the number of subjects.

3.2 Foster's Design Methodology

Foster's Design Methodology was used to guide the parallelization of the two strategies presented in this report.

Partitioning

As identified in Section 3.1, one of the most computationally expensive operations is calculating the distance between each document and each cabinet. This operation defines the primitive task of the algorithm, since each document-cabinet comparison can be performed independently and is therefore suitable for parallel execution.

Communication

After each iteration, cabinet scores must be updated globally across all processes. This synchronization step is necessary to ensure that all processes use consistent cabinet scores before the next reassignment phase; otherwise, different processes could make decisions based on outdated values. Therefore, each process computes

partial results locally, which are then combined across processes using MPI collective communication.

Agglomeration and Mapping

The primitive tasks are not distributed individually, since doing so would lead to costly communication overhead. Instead, tasks are grouped into larger units of task using two decomposition strategies: row-wise and checkerboard.

3.3 Decompositions

Row-wise Decomposition

As part of row-wise decomposition, all documents, D , are divided amongst all the available MPI processes, P . Additionally, all processes has access to the entire list of cabinets (C). Meaning that processes only need to compute the subset of documents it has.

An important understanding from this decomposition, is that there is not need to send each individual cabinet-document pair. The reason being that we can perform the calculations of the best cabinet for a given document and add them to a temporary cabinet subject score array. Here we maintain all the sum of all the document subject scores for a given cabinet and another which contains the amount of documents per cabinet. These partial results are then combined across all MPI processes in order to compute global cabinet scores.

With this approach we maintain simple and low communication overhead, and we reduce the amount of computation each process needs to perform by distributing the workload of documents evenly.

Checkerboard Decomposition

Now with the checkerboard decomposition, both documents and cabinets are divided among MPI processes, P , differently to how row-wise was. Here each process is responsible for a block/subset of documents, $D/\sqrt{P}$, and block/subset of cabinets, $C/\sqrt{P}$.

With each iteration, each process computes the distance between the local documents, D , and cabinets, C . From this we obtain a local best cabinet for each document, and these are then shared between the other processes in the same process row, as these would they share the same documents but different cabinets. After this communication, each process compares the received cabinet candidates and selects the cabinet with

the minimum distance, which becomes the final cabinet assignment for each document.

Once the final cabinet assignment is known, each process computes partial sums of subject scores and document counts for its subset of cabinets. These partial results are then shared with the other processes in the same process column, since these processes share the same subset of cabinets but different subsets of documents. This allows the cabinet scores to be re-computed correctly.

Checkerboard decomposition reduces the amount of computation per process, as we each process deals with less documents and cabinets. However, the downside is that we have much more complex communication between the different processes in order to exchange information between one another.

3.4 Complexity Analysis

Computational Complexity

The dominant computational cost comes from calculating the distance between documents and cabinets across all subjects as shown in (1)

In the row-wise decomposition, each process handles approximately D/P documents and compares them with all cabinets:

$$T_{\text{row-wise}} = O\left(\frac{D}{P} \times C \times S\right) \quad (2)$$

In the checkerboard decomposition, each process handles approximately $D/\sqrt{P}$ documents and $C/\sqrt{P}$ cabinets:

$$T_{\text{checkerboard}} = O\left(\frac{D}{\sqrt{P}} \times \frac{C}{\sqrt{P}} \times S\right) \quad (3)$$

$$T_{\text{checkerboard}} = O\left(\frac{D \times C \times S}{P}\right) \quad (4)$$

Communication Complexity

In the row-wise decomposition, cabinet score updates are shared across all processes using collective communication. The communication cost per iteration is proportional to the number of cabinets and subjects:

$$T_{\text{row-wise}} = O(C \times S \times \log P) \quad (5)$$

In the checkerboard decomposition, communication occurs across both process rows and columns. Processes exchange best cabinet assignments and recompute cabinet scores using collective communication within subgroups of size $\sqrt{P}$:

$$T_{\text{checkerboard}} = O\left(\frac{D + C \times S}{\sqrt{P}} \log \sqrt{P}\right) \quad (6)$$

4 MPI & OMP

For both the Row-based and Checkerboard-based decompositions we combined MPI with OpenMP, in which MPI handles communication across nodes while OpenMP parallelizes computation within each node, leveraging shared memory and avoiding the extra communication overhead that additional MPI processes would introduce.

4.1 Load Balancing

The parallelization approach inside each process is done the same way as in our OpenMP implementation. Each process gets assigned C cabinets and D documents, which are evenly split per thread across all parallelizable steps.

We also considered load balancing at the MPI process level. Since each process is assigned an evenly partitioned share of cabinets and documents, and the per-element computation cost is uniform (every document computes distances to the same number of cabinets), the workload is inherently balanced, making static load balancing sufficient.

4.2 Overlapping Computation and Communication

As a way to allow communication to have a small toll on our program's run times, we tried to overlap communication with computation as much as possible. This was done on both approaches for the *swaps* variable, which we do not need right away and can overlap its arrival/sending with computing the new cabinet scores.

5 Performance Results

As a way to test our two new implementations we first checked if they scaled compared to the serial implemen-

tation, as we increased the core count. We did tests for a combination of P processes and T threads. The configurations were the following: 1C*1T, 2C*2T, 4C*4T, 8C*8T and 16C*16T. This gave us the following results:

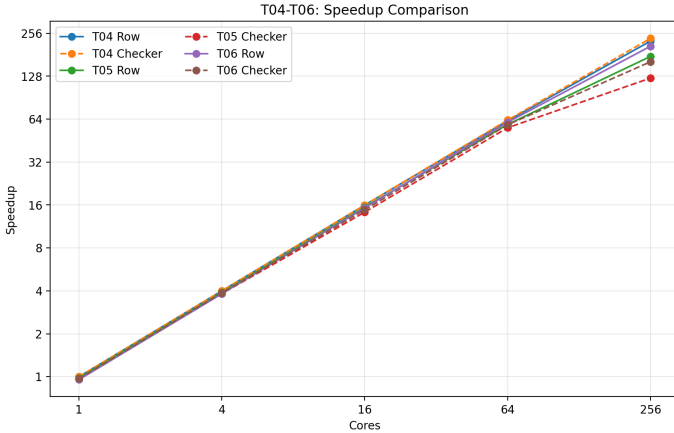


Figure 1: Observed speedup as core count increased.

Our implementation scales pretty well up to the 64 total cores. From this point onwards the $T05$ or $ex10$, and $T06$ or $ex200$ start seeing a drop in speedup. This might be due to the fact that at this scale, communication might be the biggest speedup impacting factor since there is way less computation to do per thread. This does not happen for $T04$ or $ex1000$ since there is an order of magnitude more cabinets.

We also checked if our MPI implementations scaled well as we increased only the number of processes, only 1 thread per process was used. For this tests we used a way bigger amount of processes when compared to the last tests as to test how communication affected our run times. We tested it using $T08$ or $ex10000$

Table 2: Run times and speedups for $T08$ (pure MPI, 1 thread).

Method	Cores	Time (s)	Speedup
Checkerboard	128	4580.9	1.00×
Checkerboard	256	2292.5	2.00×
Checkerboard	512	1147.3	3.99×
Checkerboard	1024	577.0	7.94×
Checkerboard	2048	289.2	15.84×
Checkerboard	4096	147.1	31.14×
Row	128	4640.5	1.00×
Row	256	2324.0	2.00×
Row	512	1165.8	3.98×
Row	1024	586.3	7.91×
Row	2048	296.2	15.67×
Row	4096	151.3	30.67×

We observed that there is no degradation in speedup as we increased process count, communication seems to barely take a toll on run times. We also noticed that surprisingly there is barely any difference when comparing our row and checkerboard based implementations which we did not expect as the checkerboard based solution only swaps messages with a way lower number of processes, $\sqrt{P}$.

Since there was no clear winner when comparing the row and checkerboard implementations, we compared both our solutions with and without the use of OpenMP with an increasing number of processes and a non 1 amount of threads. We ran tests for the following configurations: 32C*4T, 64C*4T, 128C*4T, 256C*4T, 512C*4T and 1024C*4T.

Table 3: Execution time for $T08$ with different implementations.

Cores	Chk. Hybrid	Chk. MPI	Row Hybrid	Row MPI
128	4578.2	4580.9	4628.2	4640.5
256	2291.8	2292.5	2316.0	2324.0
512	1147.5	1147.3	1159.5	1165.8
1024	575.0	577.0	581.0	586.3
2048	287.9	289.2	292.1	296.2
4096	145.2	147.1	147.9	151.3

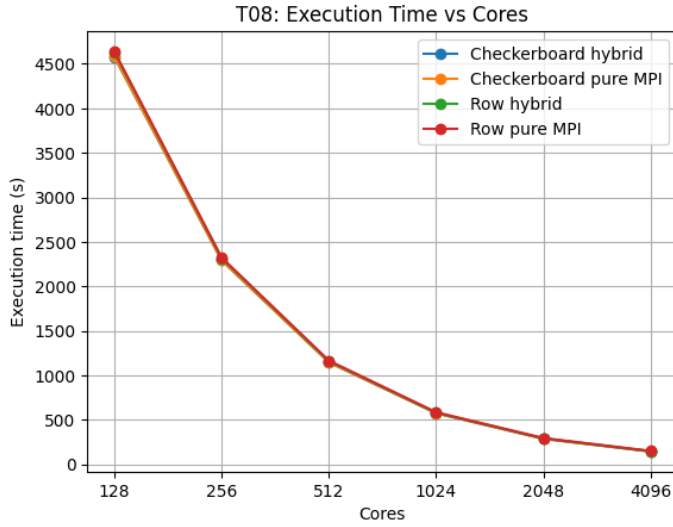


Figure 3: Observed speedup of .

We actually expected the hybrid solutions to have lower run times when compared to the pure MPI ones, since there is way less communication there, however, all implementations barely differed in run times.

6 Conclusion

Our MPI implementations achieved near-linear speedup across all tested configurations, scaling well up to 4096 cores. The row-based and checkerboard decompositions performed similarly, suggesting that for this problem, the reduced communicator size in the checkerboard approach ($\sqrt{P}$ instead of P) does not provide a significant advantage, likely because the communication volume remains small relative to computation. Similarly, the hybrid MPI+OpenMP variants did not outperform pure MPI, indicating that the overhead of additional MPI processes within a node is negligible for this workload. Overall, the dominant factor in scalability is the computation-to-communication ratio, which only becomes unfavorable for smaller problem sizes at high core counts.